EF Core Code First Workflow

In the Code First approach, you define your application models in C# first. Entity Framework Core then uses those classes to generate migrations and create or update the database schema.

Required NuGet Packages

Install the following NuGet packages to add SQL Server support and enable Entity Framework Core migration tools.

- **Microsoft.EntityFrameworkCore.SqlServer:** Adds SQL Server database support for Entity Framework Core.
- **Microsoft.EntityFrameworkCore.Tools:** Provides commands for creating and applying migrations.

Helpful Visual Studio Shortcuts

- **ctor + Tab:** Generates a constructor for the current class.
- **prop + Tab:** Creates an auto-implemented property.

Employee Model Class

The **Employee** model defines the fields that will become part of the database table. The **ID** property stores a unique identifier, while **FirstName** and **LastName** are required values. The **Email** property is optional because it uses nullable string syntax.

```
namespace EmployeeManagement.Models.Entities
{
  public class Employee
  {
    public Guid ID { get; set; }

    public required string FirstName { get; set; }
    public required string LastName { get; set; }
    public string? Email { get; set; }
  }
}
```

Application DbContext Class

The **ApplicationDbContext** class connects the application's model classes to Entity Framework Core. It inherits from **DbContext** and exposes a **DbSet** property so EF Core can create and manage the corresponding database table.

```
using EmployeeManagement.Models.Entities;
using Microsoft.EntityFrameworkCore;

namespace EmployeeManagement.Data
{
  public class ApplicationDbContext : DbContext
  {
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
      : base(options)
    {
    }

    public DbSet<Employee> Employees { get; set; }
  }
}
```

Application Configuration in appsettings.json

The **appsettings.json** file stores configuration values for the application, including logging settings, allowed hosts, and the database connection string used by Entity Framework Core.

- **Logging:** Controls how much information the application records while it runs.
- **AllowedHosts:** Defines which hosts are allowed to access the application.
- **ConnectionStrings:** Stores the database connection information used by the application.

```
{
 "Logging": {
  "LogLevel": {
   "Default": "Information",
   "Microsoft.AspNetCore": "Warning"
  }
 },
 "AllowedHosts": "*",
```

```
 "ConnectionStrings": {
  "DefaultConnection":
"Server=.;Database=EmployeesDb;Trusted_connection=true;TrustServerCertificate=true;"
 }
}
```

Understanding the DefaultConnection String

The **DefaultConnection** value tells Entity Framework Core which SQL Server database to connect to and how the application should authenticate.

- **DefaultConnection:** The name of the connection string. The application can reference this name in **Program.cs**.
- **Server=.:** Connects to the local SQL Server instance on your computer.
- **Database=EmployeesDb:** Specifies the database name. EF Core will connect to, create, or update a database named **EmployeesDb**.
- **Trusted_connection=true:** Uses Windows Authentication instead of a SQL Server username and password.
- **TrustServerCertificate=true:** Trusts the SQL Server certificate, which is commonly used during local development to avoid certificate-related connection errors.

In simple terms, this connection string tells the application to connect to the local SQL Server using the current Windows login and use the database named **EmployeesDb**.

Program.cs Configuration

The **Program.cs** file configures the application services and request pipeline. In this setup, it registers controllers, enables Swagger for API testing, connects Entity Framework Core to SQL Server, and starts the application.

Key Configuration Steps

- **Import required namespaces:** Adds access to the application database context and Entity Framework Core features.
- **Create the application builder:** Initializes the web application configuration and services.
- **Register services:** Adds controller support, Swagger/OpenAPI tools, and the database context.
- **Configure SQL Server:** Uses the **DefaultConnection** string from **appsettings.json** to connect the application to the database.

- **Configure middleware:** Enables HTTPS redirection, authorization, controller routing, and Swagger in development mode.

Program.cs Code Example

```
using EmployeeManagement.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<ApplicationDbContext>(options =>
  options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))
);

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
  app.UseSwagger();
  app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Understanding AddDbContext and UseSqlServer

This line registers **ApplicationDbContext** with the application's dependency injection container and tells Entity Framework Core to use SQL Server as the database provider.

- **builder.Services.AddDbContext<ApplicationDbContext>:** Adds the **ApplicationDbContext** class as a service so controllers and other classes can request it when they need to work with the database.
- **options:** Represents the configuration settings used by Entity Framework Core.
- **UseSqlServer(...):** Tells EF Core to connect to a SQL Server database.
- **builder.Configuration.GetConnectionString("DefaultConnection"):** Reads the **DefaultConnection** value from **appsettings.json**.

In simple terms, this code connects the application's database context to the SQL Server database defined in the **DefaultConnection** connection string.

Migration Commands

Run these commands in the Package Manager Console after creating or updating your model classes.

- **Add-Migration "InitialMigration":** Creates a migration file that records changes made to the model classes.

Summary of Core EF Concepts

Database Modeling

- **Code First Approach:** Begin by creating C# model classes, then use Entity Framework Core to create or update the database based on those classes.
- **Model Class:** Defines the structure of the data. For example, the **Employee** class includes properties such as **ID**, **FirstName**, **LastName**, and **Email**.
- **DbContext:** Acts as the main bridge between the application and the database.
- **DbSet:** Represents a database table, such as **DbSet<Employee>** for the Employees table.

Configuration and Setup

- **NuGet Packages:** Add SQL Server support and migration tools using **Microsoft.EntityFrameworkCore.SqlServer** and **Microsoft.EntityFrameworkCore.Tools**.
- **Connection String:** Tells the application where the SQL Server database is located and how to connect to it.
- **appsettings.json:** Stores application settings, including logging, allowed hosts, and database connection strings.

- **Program.cs:** Registers services, configures middleware, and connects Entity Framework Core to SQL Server.
- **Dependency Injection:** Makes **ApplicationDbContext** available throughout the application wherever database access is needed.

Database Updates

- **Migrations:** Track changes made to model classes and apply those changes to the database.
- **Add-Migration:** Creates a migration file that records model changes.
- **Update-Database:** Applies pending migrations and updates the SQL Server database schema.